

Agent 改进循环始于执行轨迹

The agent improvement loop starts with a trace

LangChain · Sam Crowder · 2026-03-31 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网页导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。

核心要点

- Agent 是围绕模型构建的系统，其中有几个可以更新的层次：模型权重、编排代码，以及上下文（提示词、指令、技能）。要知道该改什么，需要从执行轨迹中寻找证据。
- 执行轨迹可以来自任何地方：预发布环境、测试运行、基准评测、本地开发，尤其是生产环境。无论轨迹来自哪里，改进循环都是一样的。
- 这个循环需要通过评测和人工反馈丰富执行轨迹，识别失败模式，有针对性地修改，并在发布前验证。每一轮循环都会产生更好的数据，使迭代更可靠。
- LangSmith 将循环的各个环节连接起来，从第一条执行轨迹，到防止回归问题随版本发布的 CI/CD 检查关卡。

Agent 改进循环的原理很简单：获取执行轨迹，丰富轨迹信息，从中改进，再重复这一过程。本指南介绍如何在实践中做到这一点。

系统性地改进 Agent 需要一个反馈循环。你收集 Agent 的行为轨迹，通过评测和人工反馈丰富这些轨迹，识别哪里失败、为什么失败，进行有针对性的修改，并在发布之前验证这些修改是否奏效。然后，从更高的基线开始下一轮循环。

执行轨迹驱动着这个循环。轨迹可以来自许多地方，例如预发布环境、基准评测、本地开发，尤其是生产环境。重要的是收集这些轨迹，丰富其中的信息，并利用这些数据改进系统。

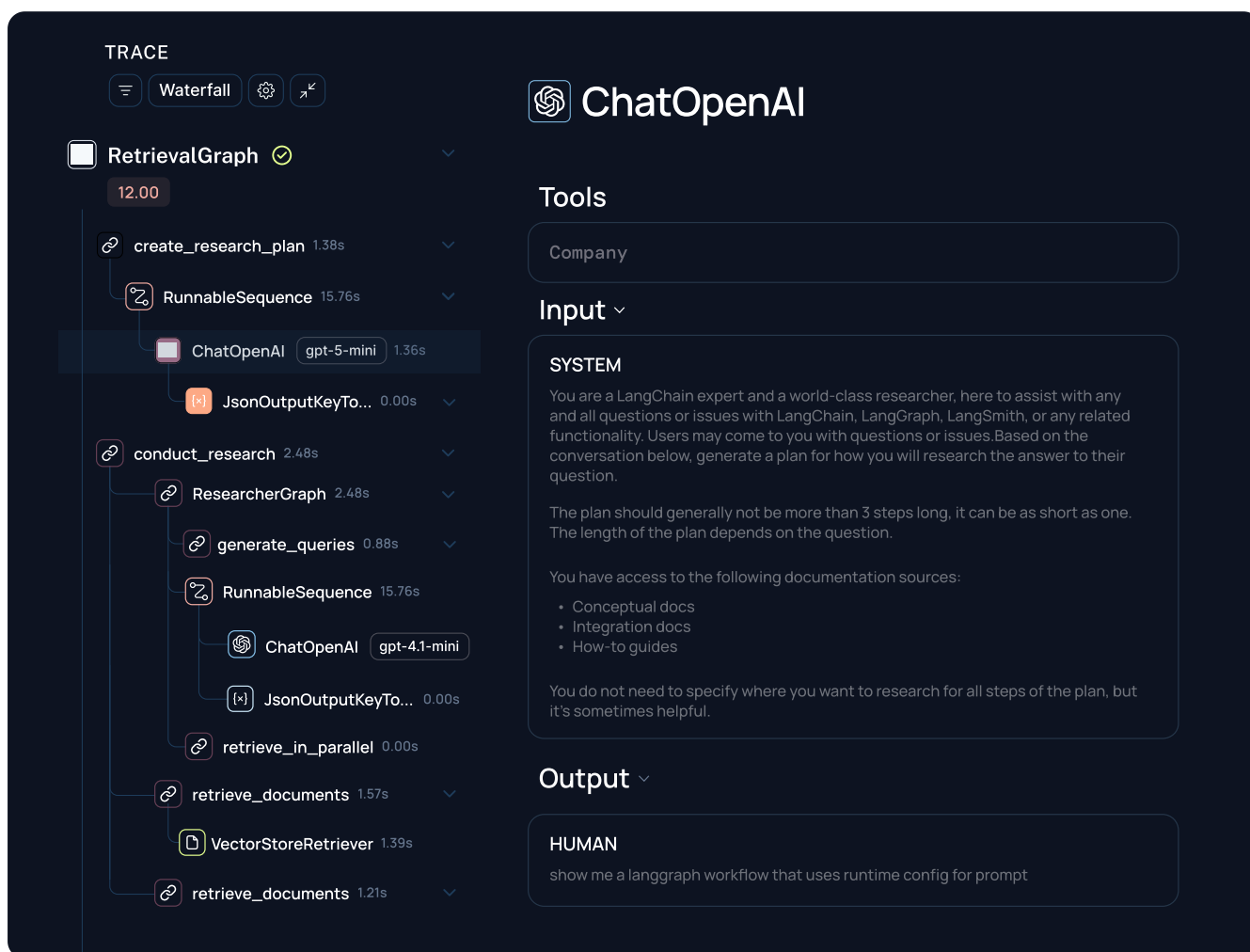
本指南将逐一介绍这一反馈循环的各个步骤。

执行轨迹是原材料

Harrison 说得很直接：“在软件中，代码记录了应用的行为；在 AI 中，承担这个角色的是执行轨迹。”

在传统应用中，代码是系统行为的权威记录。你可以阅读代码、推理其行为、据此测试，原则上能够理解系统的每一种行为。

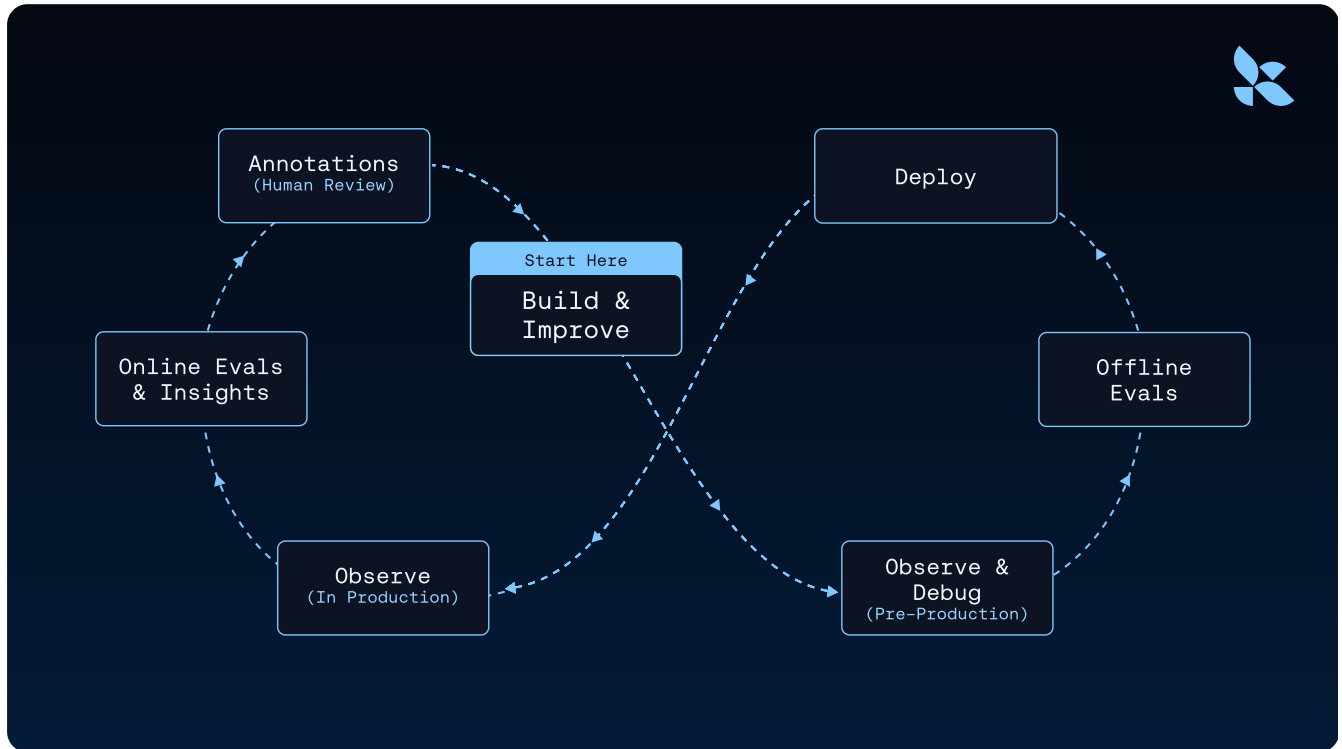
在 Agent 系统中，代码告诉你 Agent 被允许做什么；执行轨迹则告诉你，它在这次运行中，面对这个输入、处于这些条件下，实际上做了什么。



一条执行轨迹会记录 Agent 一次运行的完整执行过程：每次 LLM 调用、每次工具调用、每个检索步骤、每项中间输出，以及串联这些环节的决策序列。它记录了 Agent 在这次运行中，面对这个输入、处于这些条件下，实际采取的行动。

原始轨迹告诉你发生了什么。经过评估器评分、由审核者标注的轨迹，则为你指出该如何处理。循环的其余部分，就是建立这一信息补充层，并利用它进行有针对性、经过验证的改进。

Agent 改进循环



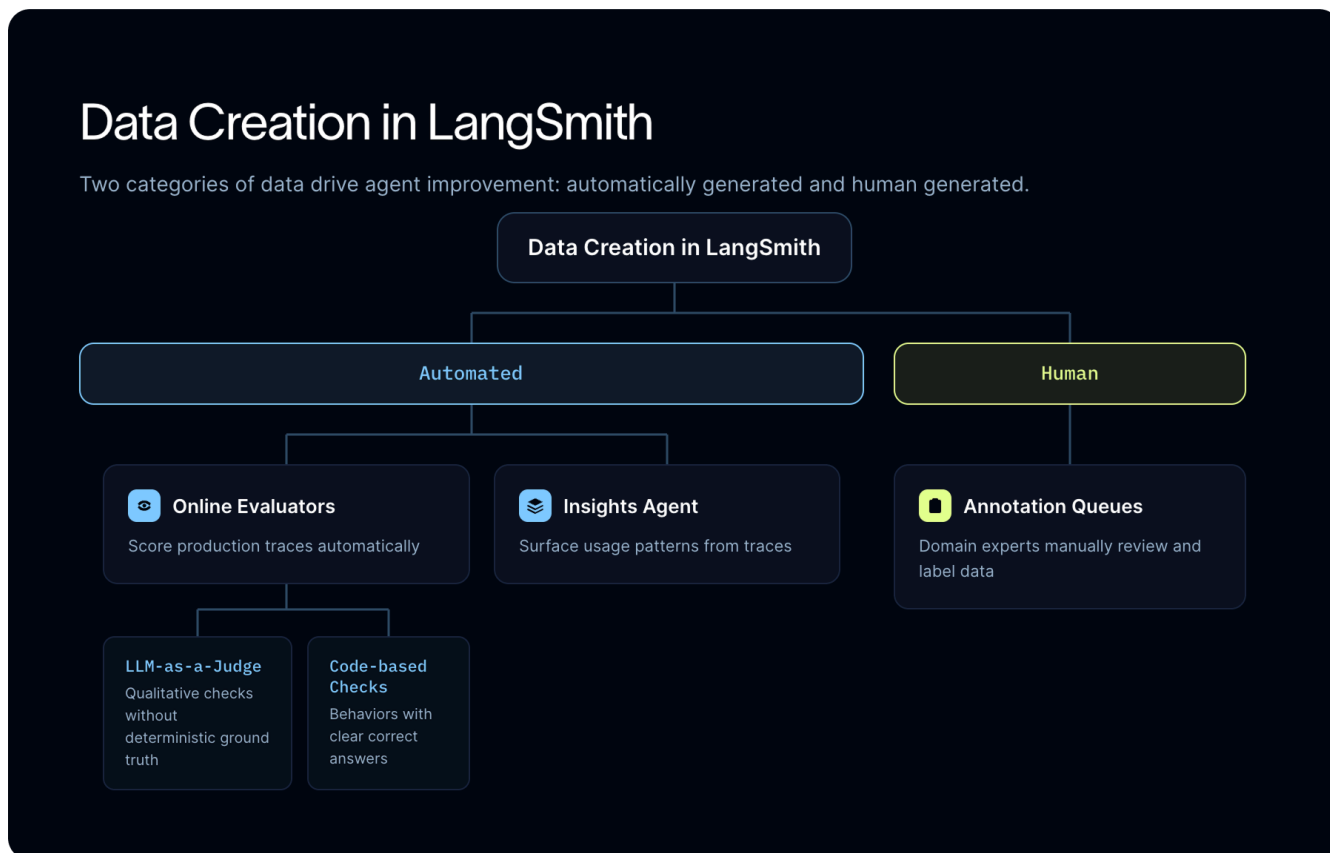
有了执行轨迹，Agent 的改进就变得具体且可重复。这个循环如下：

- 1. 构建与改进：**从你已知的内容出发：一个 Agent、一项任务，以及一个关于哪里可以做得更好的假设。开发者查看获得负面评分的轨迹，筛查失败模式，并检查导致不良结果的执行过程。他们从观察到的行为反向追查，而不是猜测该修什么。真实轨迹中呈现的失败模式，成为修改代码和提示词的依据。
- 2. 观察与调试（上线前）：**开发者在预发布环境中运行更新后的 Agent。在正式评测之前，执行轨迹就能显示修复后的行为是否符合预期。
- 3. 离线评测：**信息得到补充的轨迹被转换成可复现的测试用例。反复出现的失败模式会转化为一个评估器；一组暴露问题的真实输入会成为一个数据集。发布之前，开发者针对更新后的 Agent 运行离线评测套件，形成具体的前后对比。如果修复有效，分数就会提高；如果引入回归，问题会在影响用户之前暴露出来。通过的评测会加入长期保留的测试套件。
- 4. 部署：**修复发布后，新的执行轨迹开始积累。下一轮循环从更高的起点开始。
- 5. 观察（生产环境中）：**Agent 在生产环境中的每次运行都会产生一条轨迹，包含输入、输出、执行过程、工具调用、token 用量和延迟。这些是下一轮循环的原材料，也是判断 Agent 实际做了什么的事实依据。
- 6. 在线评测与 Insights：**原始轨迹带上更多信号后，就会变得更有用。自动评估器持续为输出评分；Insights 报告则从大量轨迹中揭示使用模式、失败模式和边界情况。
- 7. 标注：**人工审核者对选中的轨迹添加评分、纠正和评论。每一层补充信息都为原始行为记录增加上下文，形成标注数据，并反馈到下一轮构建过程。

这一循环会产生累积效应，因为每轮都会生成更好的数据。更多轨迹意味着更多失败模式的实例；更多实例意味着更精确的评测；更精确的评测意味着更可靠的迭代。

LangSmith 如何从执行轨迹中自动生成数据

驱动 Agent 改进的数据分为两类：自动生成的数据和人工生成的数据。LangSmith 可以帮助你生成这两类数据。要自动生成数据，可以使用在线评估器和 Insights Agent。



在线评估器

在线评估器会自动处理生产环境中的轨迹，按照可配置的质量标准为输出评分。你可以配置它们处理全部轨迹、抽样得到的子集，或按特定条件筛选出的子集。

评分方法取决于你要评估什么。

对于没有确定性标准答案的定性维度，例如有用性、语气、相关性、政策遵循情况和事实上的可信程度，可以使用 LLM 作为裁判。评估器调用 LLM，检查的不仅是最终回答，还有完整执行轨迹：Agent 是否使用了正确的工具、调用顺序是否正确、参数是否正确？

对于有明确正确答案的行为，可以使用基于代码的检查。Schema 校验、精确匹配条件、格式符合性、业务规则遵循情况和工具使用的正确性，都可以用确定性方式评估；这比交给 LLM 裁判更快，也更便宜。

持续洞察与报告

coding-agent-insights

> Category

Description

Runs

Effectiveness

hallucination

usage

> Dev Experience	Tutorials, examples, debugging, prompts, and structured-output patterns for developer productivity.	126	6.53	0.73	1.00
> Platform	Deployment, self-hosting, API keys, tokens, infrastructure configuration, and system architecture.	79	6.63	0.29	1.00
> Orchestration	Graph routing, async runtimes, streaming, multi-agent flows, and system patterns.	53	4.3	0.29	
> Indexing	Embeddings, vector stores, similarity search, retrieval systems, and citation strategies.	37	8.00	0.63	
> Frontend	UI frameworks, Streamlit/React issues, UX, state management, and application integration.	18	8.30	0.60	
> Security	Compliance, data protection, SOC2, GDPR, licensing, and regional hosting controls.	15	8.88	0.50	
> Evaluation	Automated evaluation, tracing, metrics, result sharing, and model performance assessment.	12	6.71	0.25	

LangSmith 的 Insights Agent 会对生产轨迹进行自动聚类，以发现使用模式、失败模式和边界情况。这与监控有所不同：你不是在追踪事先定义好的指标，而是在发现此前不知道应该关注的模式。

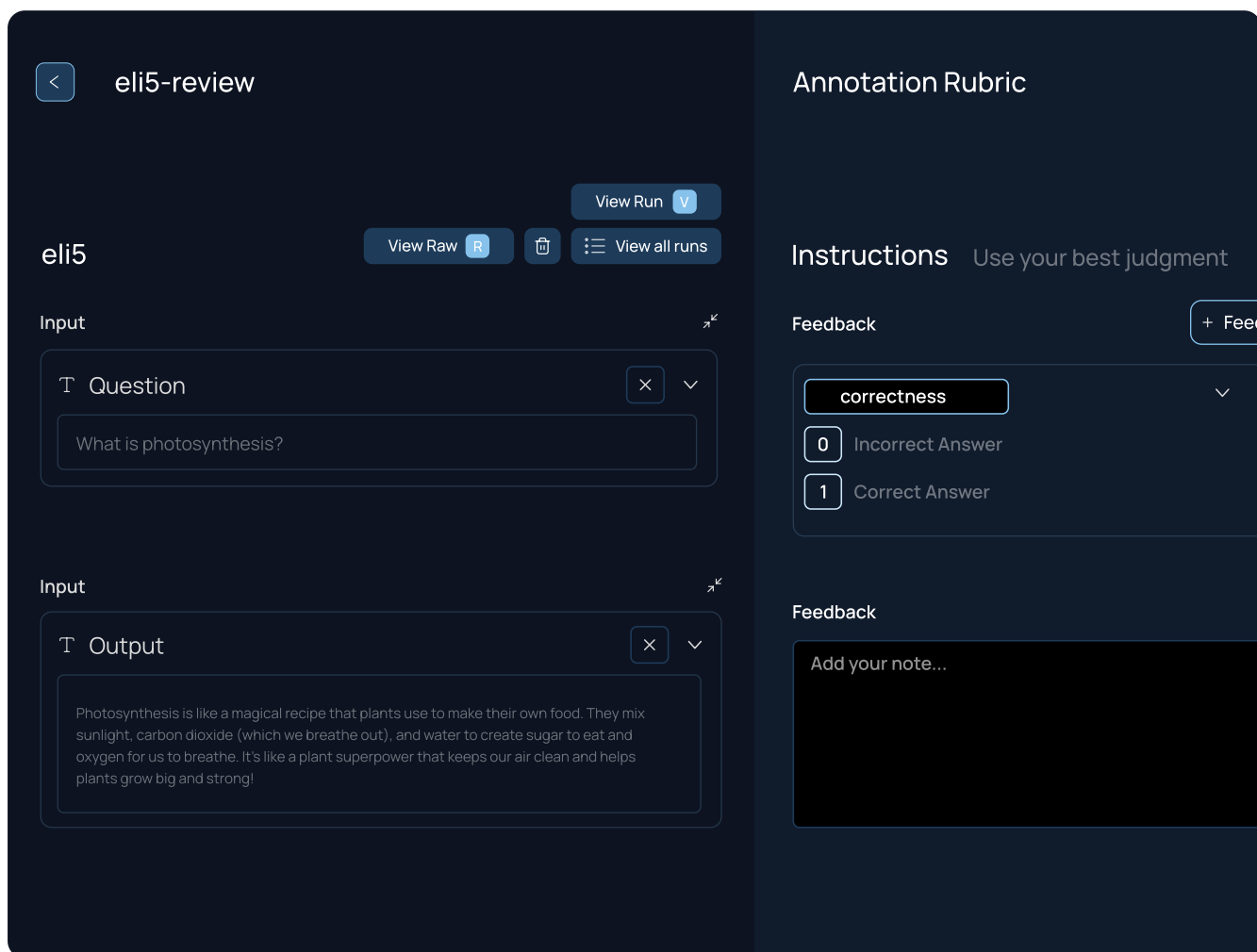
一个负责面向客户的 Agent 的团队可能会问：“用户究竟想用这个 Agent 做什么？”Insights Agent 可以分析数千条轨迹，按意图分组，并找出最主要的类别，其中也包括此前没有人预料到的类别。对收到负面反馈或低分的轨迹进行同样的分析，则可以揭示 Agent 经常在哪些方面表现不足，以及原因是什么。

自动化的边界：人工判断仍然重要

自动评估器与自动洞察能够很好地扩展到大规模场景，但无法取代人工判断。

有些 Agent 行为只有具备领域知识的人才能评估。法律研究 Agent 引用听起来可信、实际上不准确的判例，可能骗过 LLM 裁判；医疗信息 Agent 提供技术上正确、但临床上不适当的指导，在自动检查中看起来也没有问题。专业领域里细微的失败，需要真正理解何谓“正确”的审核者来识别。

这时就需要标注队列。



团队可以通过筛选条件，将选中的生产轨迹送入标注队列，例如自动评分较低的轨迹、来自特定功能领域的轨迹，以及被最终用户点踩的轨迹。审核者可以查看完整上下文，并添加评分、纠正、评论以及修改后的输出。

实际工作中，团队使用标注队列主要有四种方式：

- **校准在线评估器：**审核者为轨迹打标签，用于校准 LLM 裁判。当人工审核者与自动评估器意见不一致时，这些标注样本可以帮助调整评分器，直到评分能够反映人的判断。
- **为离线数据集建立标准答案：**审核者标注一条轨迹应该得到的正确最终输出。这些内容成为离线评测套件中的预期答案，让你可以用生产环境的输入检验后续版本的正确性。
- **为开放式输出评分：**当不存在唯一正确答案时，审核者标注定义优质回答的标准。对于过于细腻、无法通过精确匹配检查的维度，这类结构化反馈可以成为评估器的依据。
- **自然语言标注：**审核者为轨迹添加自由形式的评论和纠正。这些内容会进入 Insights Agent 的分析，从而揭示仅凭分数无法发现的模式。

第一种用途值得与其余几种区分开来。用于校准在线评估器的标注，会改善实时监控，使持续进行的自动评分更加准确。另外三种用途主要是构建离线数据集，建立标准答案和质量标签，使你能在发布之前测试修复。

实际工作中常见两类审核者。

- **通用审核者：**外包人员、标注人员和客户成功团队可以评估表层的质量信号。他们判断回答是否有帮助、相对于可见信息是否准确，以及语气是否恰当。
- **领域专家：**产品经理、领域专家（SME），以及 Agent 所服务领域的专业人员，可以判断 Agent 在具体上下文中的行为是否正确，包括自动化完全无法发现的失败。

在现阶段，你往往仍然需要让人参与循环。

如何使用补充了信息的轨迹：构建与改进

补充了信息的执行轨迹，是理解 Agent 在哪些地方反复失败的原材料。

多条轨迹中共同出现的模式，比任何单个例子都更有助于采取行动。你会开始发现，Agent 总是误解某类查询，或者总是在某种上下文中选错工具。

只抽查个别运行，很难形成对模式的认识。这需要大规模、标注一致、来自真实生产行为的数据。

具体如何修复，取决于轨迹揭示了什么。如果 Agent 对某类查询选错工具，可能需要更新工具说明，或者增加路由逻辑。如果它在多步骤任务进行到一半时推理偏离方向，可能需要约束更明确的系统提示词，或把任务拆成更小、更聚焦的步骤。如果输出在事实层面正确，却没有回应用户的真实意图，这通常是提示词层面的问题，需要在指令中明确什么才是“好”。有时，轨迹还会揭示结构性问题：Agent 需要一种完全不同的工具，或者 workflows 需要在特定决策点设置人工参与的检查环节。

每一项这样的改动，都依据具体观察到的行为，而不是假想的失败模式。开发者之所以重写提示词，是因为他们能明确看到哪些轨迹失败了、如何失败，以及标注反馈如何解释失败原因。

随后，离线评测让这些提示词和代码改动的效果变得可以衡量。

将生产环境中的失败转化为离线评测

确定需要修复什么之后，你需要一种方法来测试修复是否真正奏效。这就是离线评测发挥作用的地方。

这些评测所用的数据集应来自生产环境：真实轨迹、真实查询和真实失败。

评测实际衡量什么，取决于标注工作产出了什么。有两种不同的方法：

- **依据标准答案判断正确性：**如果审核者已经标注了某条轨迹的正确最终输出，就可以直接测试正确性。在数据集上运行改进后的 Agent，将其输出与标注的标准答案比较。如果修复有效，分数会提高；如果引入回归，评测会在问题影响用户之前发现它。
- **基于标准评分：**并非每个输出都有唯一正确答案。对于开放式任务，审核者标注的是定义优质回答的标准，而非回答本身。离线评测使用这些标准对更新后版本的输出评分，让你无需精确匹配，也能衡量相关性、完整性或语气等维度上的改进。

每一种被你编码为评测用例的失败模式，都应该永久保留在测试套件中。这样既能长期记录 Agent 已经学会处理的问题，也能形成一道关卡，确保未来的改动不会重新引入已经解决的问题。

结合在线评测与离线评测

Online + Offline Evals Together		
Two evaluation modes create a bridge between production observation and safe iteration — turning agent improvement into something measurable.		
	 ONLINE EVALS	 OFFLINE EVALS
Environment	Production — monitors live agent behavior continuously	Development — controlled experiments on curated datasets
Purpose	Catch quality drift, surface emerging failure patterns, flag traces for human review	Validate a change before it ships — confirm whether a fix actually addresses the problem
Timing	Continuous — always running against live traffic	Pre-deployment — run before any change reaches production
Key question	"What's going wrong?"	"Does the fix actually work?"
Output	Alerts, flagged traces, quality scores over time	Pass/fail results, comparative scores across versions
Applies to	All live interactions — samples or full coverage	Prompt changes, model updates, workflow modifications, architecture changes

在线评估器持续监控实际运行行为。它们捕捉质量漂移，发现新出现的失败模式，并标记需要人工审核的轨迹。但它们不能让你在发布之前验证某项改动。

离线评测可以解决这个问题。它是在开发阶段、任何改动进入生产环境之前，在经过整理的数据集上运行的受控实验。

两者结合，就在生产观察与安全迭代之间架起了一座桥梁。在线评测告诉你哪里出了问题；离线评测确认你的修复是否真正解决了问题。

每一种被你编码为评测用例的失败模式，都应该永久保留在测试套件中。这样既能长期记录 Agent 已经学会处理的问题，也能形成一道关卡，确保未来的改动不会重新引入已经解决的问题。

每次提示词修改、模型更新、工作流调整或架构变更，都应在发布之前使用累积起来的评测套件进行评测。持续运行相同的评测集，比较不同版本和模型配置的分数的，就能让改进循环变得可以衡量。你可以证明，每轮迭代都产生了更好的 Agent，而不只是一个不同的 Agent。

让编码 Agent 参与循环

改进循环正在变得更加自动化，而执行轨迹记录仍然处于其核心位置。

LangSmith CLI 和 Skills 使编码 Agent 可以直接从终端以专家级方式访问 LangSmith 数据。配备 LangSmith Skills 后，在我们的评测集上，Claude Code 的表现从 17% 提升到了 92%。

实际操作时，开发者可以指示编码 Agent 拉取最近 30 天的生产轨迹，筛选被点踩的轨迹，识别它们体现的失败模式，根据这些例子起草评测，并提出相应的提示词或代码修改。所有这些工作都可以在一次终端会话中完成，并以真实行为数据为依据。

但没有轨迹数据的编码 Agent，只能依据不完整的信息进行修改。它会提出从代码审查角度看似合理、却没有针对实际失败模式的修复，因为它看不到导致失败的执行过程。使用补充了信息的轨迹工作的编码 Agent，所依据的信息与资深工程师会使用到的信息相同。

执行轨迹记录是 Agent 改进循环的基石

可靠的 Agent 并不是靠调试个别轨迹构建起来的，而是来自以轨迹为中心的改进循环。

这个循环始于轨迹记录，也回到轨迹记录。每个评估器都针对轨迹运行；每项标注都附着于一条轨迹；每个离线数据集都由轨迹构建；每次回归测试都针对真实轨迹中观察到的情况进行验证。提出下一项修复的编码 Agent，也要读取轨迹来完成这项工作。

因此，执行轨迹记录不仅是一种调试工具。它是使整个改进循环成为可能的基本构件，是所有评测、所有人工反馈和所有系统性改进的基础。

循环始于一条执行轨迹。下一轮循环，则始于反馈回来的那条轨迹。

延伸阅读

- 《Agent 可观测性如何支撑 Agent 评测》
- 《Agent 上线之前，你无法知道它究竟会做什么》
- LangSmith 文档：《离线评测类型》
- LangSmith 文档：《在线评测类型》
- LangSmith 文档：《标注队列》
- LangSmith 文档：《Insights》